

```

// Component state-ээс хамааралгүй туслах (pure) функцвд + upload helper

import { supabase } from './supabase';

// ЗАСВАР #11: upload хийхийн өмнө файлын төрлийг шалгах нэг цэгтэй функц
// (өмнө нь <input accept="image/*"> л байсан бөгөөд энэ нь зөвхөн UI-д зориулсан
// зөвлөмж тул хэрэглэгч ямар ч файл сонгож upload хийж болдог байсан).
// ЗАСВАР #17: хэмжээний (8MB) хязгаарлалтыг хассан – жинхэнэ hosting (Supabase
// Storage) холбогдсон тул бүлгийн өндөр чанартай том зургийг хориглох шаардлагат
// ЗАСВАР #181 (код шинжилгээ): "image/*" бvгдийг зөвшөөрдөг байсан тул
// image/svg+xml ч нэвтэрдэг байв – SVG дотор <script> байж болох тул (R2-ийн
// public URL-ыг шууд нээхэд ажиллана) stored XSS эрсдэлтэй. Зөвшөөрөгдсөн
// төрлийг сервер (upload-to-r2 edge function) талын allowlist-той адилхан болгов
const ALLOWED_IMAGE_TYPES = ['image/jpeg', 'image/png', 'image/webp', 'image/gif']
export const validateImageFile = (file) => {
  if (!file) return 'Файл сонгогдоогүй байна.';
  if (!ALLOWED_IMAGE_TYPES.includes(file.type)) return 'Зөвхөн зургийн файл (jpg,
  return null;
};

// ЗАСВАР #94: зургийн upload-ыг Supabase Storage-с Cloudflare R2 руу шилжүүлэв
// (upload-to-r2 edge function-оор дамжуулж, Secret Access Key browser талд гарда
export const uploadToR2 = async (file, path) => {
  const { data: { session } } = await supabase.auth.getSession();
  const formData = new FormData();
  formData.append('file', file);
  formData.append('path', path);
  const res = await fetch(`${import.meta.env.VITE_SUPABASE_URL}/functions/v1/uplo
    method: 'POST',
    headers: { Authorization: `Bearer ${session?.access_token}` },
    body: formData,
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Upload алдаа гарлаа');
  return data.publicUrl;
};

// ЗАСВАР #125: R2 дээрх бодит файлыг устгах (зөвхөн админ, серверт дахин шалгагд
// urls нь uploadToR2-с буцсан бvтэн public URL-ууд байна – path-ыг нь server
// талд R2_PUBLIC_BASE_URL-аар нь тайрч тооцно.
export const deleteFromR2 = async (urls) => {
  if (!urls || urls.length === 0) return;
  const { data: { session } } = await supabase.auth.getSession();
  const res = await fetch(`${import.meta.env.VITE_SUPABASE_URL}/functions/v1/uplo

```

```

    method: 'DELETE',
    headers: { Authorization: `Bearer ${session?.access_token}`, 'Content-Type':
    body: JSON.stringify({ urls })},
  });
const data = await res.json();
if (!res.ok) throw new Error(data.error || 'Устгах алдаа гарлаа');
return data;
};

// "2026 оны 6-р сарын 25" маягийн огноо
export const formatMnDate = (dateStr) => {
  const d = new Date(dateStr);
  return `${d.getFullYear()} оны ${d.getMonth() + 1}-р сарын ${d.getDate()}`;
};

// ЗАСВАР #68: "2026.07.13" маягийн цэвэрхэн тоон огноо (бүлгийн жагсаалтад ашигл
export const formatNumericDate = (dateStr) => {
  const d = new Date(dateStr);
  const mm = String(d.getMonth() + 1).padStart(2, '0');
  const dd = String(d.getDate()).padStart(2, '0');
  return `${d.getFullYear()}.${mm}.${dd}`;
};

// ЗАСВАР #139: манганы үзэлт (views) хиймлээр өсгөхөөс сэргийлэх зорилгоор
// increment_manga_views RPC-д зочин (нэвтрээгвй) хэрэглэгчийг ялгах тогтвортой
// (browser-д хадгалагдсан) key дамжуулна – нэвтэрсэн бол сервер талд auth.uid()
// ашиглах тул үүнийг үл хэрэглэнэ, харин зочинд өөр аргагвй тул хэрэгтэй.
export const getAnonViewerKey = () => {
  try {
    let key = localStorage.getItem('anon_viewer_key');
    if (!key) {
      key = crypto.randomUUID();
      localStorage.setItem('anon_viewer_key', key);
    }
    return key;
  } catch {
    return '';
  }
};

// Үлдсэн хугацааг "2 өдөр 3 цаг" маягаар
export const formatRemaining = (ms) => {
  if (ms <= 0) return '';
  const mins = Math.ceil(ms / 60000);
  const days = Math.floor(mins / 1440);
  const hours = Math.floor((mins % 1440) / 60);
  const m = mins % 60;

```

```

    if (days > 0) return `${days} өдөр ${hours} цаг`;
    if (hours > 0) return `${hours} цаг ${m} мин`;
    return `${m} мин`;
};

```

```

// ЗАСВАР #163: "Хуваах" горим — 4000px-ээс өндөр (урт) зургийг тэр өндрөөр нь
// (өргөнийг вл өөрчлөн) олон хэсэг болгож таслана. 4000px-ээс богино/тэнцүү
// зургийг вл хөндөж, нэг элементтэй массив (өөрөө) хэвээр буцаана. Зургуудыг
// НЭГ НЭГЭЭР нь (Promise.all биш) дараалуулж дуудахыг зөвлөнө — эс бол олон
// том зургийг зэрэг декодлож санах ойн ачаалал үүсгэнэ (өмнөх ЗАСВАР #163-ийн
// urt зургийн crash-тай адил асуудал).
// ЗАСВАР #193 (код шинжилгээ): маш урт (өндөр нягтралтай) зураг оруулахад
// browser tab бvхэлдээ "унаад" (crash/freeze) байсан гомдол ирсэн — үүний
// шалтгаан нь createImageBitmap(file) нь эх зургийг ПИКСЕЛИЙН хэмжээгээр нь
// (файлын MB биш) бvхэлд нь санах ойд decode хийдэгт байна: жишээ нь
// 4000x60000 пиксел зураг ойролцоогоор 1GB+ санах ой шаардана. Үүнийг бvрэн
// арилгах боломжгvй (canvas ашиглахын тулд заавал decode хийх ёстой) тул,
// хамгийн багадаа browser-ийг найдваргvй байдалд оруулахын оронд ойлгомжтой
// алдаа vзvvлж, эх зургийг жижигрvvлж дахин оруулахыг санал болгоно.
const MAX_SAFE_PIXELS = 120_000_000; // ~120 megapixel — ердийн урт вэбтvн (webto
export const splitTallImageFile = async (file, maxHeight = 8000) => {
    const bitmap = await createImageBitmap(file);
    const { width, height } = bitmap;
    // ЗАСВАР #202: 120MP (MAX_SAFE_PIXELS) хязгаарыг энэ функцээс хассан — урт
    // зургийг хэмжээнээс үл хамааран автоматаар хэсэглэнэ. (Санамж: маш том эх
    // зургийг createImageBitmap бүхэлд нь санах ойд задалдаг тул upload хийж буй
    // төхөөрөмж дээр санах ой ихээр зарцуулж болзошгүй.)
    if (height <= maxHeight) {
        bitmap.close?();
        return [file];
    }

    const ext = (file.name.split('.').pop() || 'jpg').toLowerCase();
    const mimeType = file.type || (ext === 'png' ? 'image/png' : 'image/jpeg');
    const baseName = file.name.replace(/\.[^.]*/, '');
    const canvas = document.createElement('canvas');
    canvas.width = width;
    const ctx = canvas.getContext('2d');

    const pieces = [];
    let y = 0;
    let partIndex = 1;
    while (y < height) {
        const pieceHeight = Math.min(maxHeight, height - y);
        canvas.height = pieceHeight;
        ctx.clearRect(0, 0, width, pieceHeight);
        ctx.drawImage(bitmap, 0, y, width, pieceHeight, 0, 0, width, pieceHeight);

```

```

// eslint-disable-next-line no-await-in-loop
const blob = await new Promise(resolve => canvas.toBlob(resolve, mimeType, 0.
// ЗАСВАР #191 (код шинжилгээ): маш том canvas дээр browser toBlob-оор null
// буцааж болно (жишээ нь санах ой хврэлцэхгвй вед) – шалгахгвй бол
// new File([null], ...) гэсэн эвдэрсэн (0 байттай) файл vvсгэдэг байв.
if (!blob) throw new Error('Зургийг хэсэглэхэд алдаа гарлаа (санах ой хврэлцэ
pieces.push(new File([blob], `${baseName}-p${partIndex}.${ext}`, { type: mime
y += pieceHeight;
partIndex += 1;
}
bitmap.close?();
return pieces;
};

// ЗАСВАР #173: зургийг өгөгдсөн тэгш өнцөгт хэсгээр нь таслана. rect нь эх
// зургийн БОДИТ (natural) пикселийн нэгжээр өгөгдсөн байх ёстой.
export const cropImageFile = async (file, rect) => {
  const bitmap = await createImageBitmap(file);
  // ЗАСВАР #193: splitTallImageFile-тэй адил шалтгаанаар (эх зургийг бvхэлд нь
  // decode хийхэд browser tab унах эрсдэлтэй) хэт өндөр нягтралтай эх зургийг
  // тайрахаас өмнө шалгана.
  if (bitmap.width * bitmap.height > MAX_SAFE_PIXELS) {
    bitmap.close?();
    throw new Error(`Зураг хэт өндөр нягтралтай (${bitmap.width}x${bitmap.height}
  }
  const canvas = document.createElement('canvas');
  canvas.width = Math.round(rect.width);
  canvas.height = Math.round(rect.height);
  const ctx = canvas.getContext('2d');
  ctx.drawImage(bitmap, rect.x, rect.y, rect.width, rect.height, 0, 0, canvas.wid
  bitmap.close?();
  const mimeType = file.type || 'image/jpeg';
  const blob = await new Promise(resolve => canvas.toBlob(resolve, mimeType, 0.92
  // ЗАСВАР #191 (код шинжилгээ): доорх splitTallImageFile-тэй адил шалтгаанаар
  if (!blob) throw new Error('Зургийг тайрахад алдаа гарлаа (санах ой хврэлцэхгвй
  return new File([blob], file.name, { type: mimeType });
};

// ЗАСВАР #146: "цаг:минут:секунд" (жишээ нь 12:15:28) маягийн цэвэрхэн тоон
// countdown формат – хуваарийн хуудсанд секунд тутам шинэчлэгдэж харагдана
export const formatCountdownClock = (ms) => {
  if (ms <= 0) return '00:00:00';
  const totalSec = Math.floor(ms / 1000);
  const hh = Math.floor(totalSec / 3600);
  const mm = Math.floor((totalSec % 3600) / 60);
  const ss = totalSec % 60;
  const pad = n => String(n).padStart(2, '0');

```

```
    return `${pad(hh)}:${pad(mm)}:${pad(ss)}`;
};
```